

## O que é Modularização e o que são Funções?

**Definição:** Funções são blocos de código nomeados criados para realizar uma tarefa específica.  
PY

### Vantagens Principais:

**Reutilização de Código:** Escreve-se a lógica uma única vez e chama-se quantas vezes for necessário.  
PY

**Modularidade:** Divisão de problemas complexos em partes menores e organizadas.  
PY

**Manutenção Simplificada:** Alterar uma lógica em um único ponto reflete em todo o programa.  
PY

## 2. Tipos de Funções (Código do Script Explicado)

### Tipo 1: Função sem parâmetro e sem retorno

Definida com a palavra-chave `def`, parênteses vazios e sem o uso da instrução `return`.  
Ideal para ações fixas, como exibição de alertas ou menus.

PY + 1

Python

```
# Definindo a função
def mensagem():
    print('Texto da função')

# Chamando/Executando a função 3 vezes
mensagem()
mensagem()
mensagem()
```

### Tipo 2: Função com passagem de parâmetros (sem retorno)

Parâmetros funcionam como variáveis locais de entrada que recebem dados quando a função é executada.

PY

Python

```
# Função com 1 parâmetro
def mensagem(texto):
    print(texto)
```

```

mensagem('texto 1')
mensagem('texto 2')
mensagem('texto 3')

# Função com 2 parâmetros de entrada
def soma(a, b):
    print(a + b)

soma(2, 3) # Exibe 5 na tela
soma(3, 3) # Exibe 6 na tela

```

### Tipo 3: Função com parâmetros e valor de retorno

O comando `return` envia o resultado da função de volta para onde ela foi chamada, permitindo guardar esse valor em uma variável ou usá-lo em outros cálculos.

PY

Python

```

def soma(a, b):
    return a + b

# Armazenando o retorno da função na variável 'r'
r = soma(3, 2)
print(r) # Exibe 5

```

### Tipo 4: Parâmetros Opcionais (Default) e Docstrings

Podemos definir valores padrões para parâmetros (tornando-os opcionais na chamada) e adicionar documentações (docstrings) entre aspas triplas `'''`.

PY

Python

```

def calcula_energia_potencial_gravitacional(m, h, g = 10):
    '''
    Calcula a energia potencial gravitacional
    Argumentos:
    m: massa, entrada como uma variável float
    h: altura, entrada como uma variável float

    Argumento opcional:
    g: aceleração gravitacional, com valor default de 10
    '''

```

```

'''
e = g * m * h
return e

# Uso sem passar o parâmetro 'g' (assume o valor padrão 10)
energia1 = calcula_energia_potencial_gravitacional(30, 12)
print(energia1) # Retorna 3600.0

# Uso sobrescrevendo o parâmetro 'g' para 9.8
energia2 = calcula_energia_potencial_gravitacional(30, 12, 9.8)
print(energia2) # Retorna 3528.0

# Exibição da documentação da função no console/ambiente
help(calcula_energia_potencial_gravitacional)

```

### 3. Lista de Exercícios Práticos para os Alunos

#### Exercício 1 (Sem parâmetro):

Crie uma função chamada `saudacao()` que exiba na tela a mensagem "Seja bem-vindo ao curso de Python!" .

#### Exercício 2 (Com parâmetros):

Crie uma função chamada `calcular_media(nota1, nota2)` que receba duas notas, calcule a média aritmética e mostre a mensagem "A média é: [VALOR]" .

#### Exercício 3 (Com retorno):

Crie uma função chamada `par_ou_impar(numero)` que receba um número inteiro e retorne o texto 'Par' se o número for par e 'Ímpar' se for ímpar.

#### Exercício 4 (Parâmetro Padrão e Docstring):

Crie uma função chamada `desconto_produto(preco, taxa_desconto = 0.10)` que calcule o valor final de um produto aplicando um desconto padrão de 10% (0.10).

Inclua uma **Docstring** explicando o que a função faz e os parâmetros aceitos.

PY

Teste a função informando apenas o preço e depois informando o preço e uma taxa de desconto personalizada de 20% (0.20).